

Contents

Chapter 25: Build	2
From Specification to Working AI System	2
1. The New Development Loop	2
2. Your Role Changes	3
The Coding Agent	3
You	3
3. Start From the Specification	4
4. Establish the Repository	4
5. Build the Vertical Slice First	5
6. Why Vertical Slices Matter	6
7. Agent Delegation	6
8. Give the Agent Boundaries	7
9. Observe the Agent's Work	8
10. Test Continuously	8
11. Deterministic Tests Are Not Enough	9
12. Build the Evaluation Harness During the Build	10
13. The Agent Should Test Its Own Work	10
14. Human Evaluation Remains Necessary	11
15. Architecture Review	11
16. Avoid Premature Generalization	12
17. Manage Context Deliberately	13
18. Git Is Part of the Control System	13
19. The Build Loop	14
20. A Practical Build Sequence	14
Phase 1 — Repository setup	14
Phase 2 — Core data model	15
Phase 3 — Vertical slice	15
Phase 5 — Context engineering	15
Phase 6 — Agent behavior	15
Phase 7 — Verification	15
Phase 8 — Evaluation	15
Phase 9 — UX	16
Phase 10 — Hardening	16
21. Cost Is a First-Class Build Constraint	16
22. Reliability Is a System Property	17
23. Failure Handling	17
24. Human Control	18
25. The Agent as a Junior Engineer	18
26. The Agent as a Compiler for Intent	19
27. Build Metrics	20
28. The Real Objective of Chapter 25	20
29. Chapter 25 Deliverable	21
30. The Chapter 25 Build Checklist	22
31. Key Takeaways	22

Chapter 25: Build

From Specification to Working AI System

Chapter 24 defined the product.

Chapter 25 builds it.

This is the point at which the AI engineering workflow changes from **design and specification** to **execution**.

The central objective is not to prove that you can manually write every component.

It is to learn how to operate effectively in a development environment where an AI coding agent performs a substantial portion of the implementation.

Your role changes accordingly.

You are no longer primarily the implementer.

You become:

Product Owner + Architect + Evaluator

The coding agent becomes the primary implementation engine.

The engineering challenge is therefore no longer simply:

“Can I write the software?”

It becomes:

“Can I direct, constrain, inspect, test, and evaluate an AI system that writes the software?”

This is a fundamentally different engineering skill.

1. The New Development Loop

Traditional software development often looks like:

Requirement → Human Code → Test → Debug

AI-assisted development introduces another layer:

Specification → Coding Agent → Implementation → Tests → Evaluation → Revision

The agent becomes part of the development system.

A more complete formulation is:

$S \rightarrow A \rightarrow I \rightarrow V \rightarrow F \rightarrow A'$
--

where:

- S = specification,
- A = coding agent,
- I = implementation,
- V = verification,
- F = feedback,
- (A') = revised agent execution.

The human is responsible for controlling this loop.

2. Your Role Changes

The Chapter 25 role division should be explicit.

The Coding Agent

The agent is responsible for:

- writing code,
- creating files,
- implementing APIs,
- implementing UI components,
- writing tests,
- integrating libraries,
- fixing straightforward bugs,
- refactoring,
- generating documentation,
- running development commands.

You

You are responsible for:

- product scope,
- architecture,
- system boundaries,
- requirements,
- tradeoffs,
- security constraints,
- acceptance criteria,
- evaluation,
- prioritization,
- deciding what is correct,
- deciding what should be changed.

This produces a new division of labor:

Human: What + Why + Constraints

Agent: How + Implementation

This division is not absolute. You may still write code when useful.

But the goal of the exercise is to learn to operate effectively at the higher level of abstraction.

3. Start From the Specification

Do not begin Chapter 25 by opening an empty editor and asking:

“What should I build?”

You already answered that question on Chapter 24.

The specification is the source of truth.

The initial coding-agent prompt should therefore communicate:

- product objective,
- target user,
- workflow,
- requirements,
- architecture,
- interfaces,
- evaluation criteria,
- constraints,
- MVP scope.

Conceptually:

Specification $\rightarrow$ Agent Context $\rightarrow$ Repository

The specification should be treated as an engineering contract.

If the agent begins making product decisions that were already resolved, the development process is drifting.

4. Establish the Repository

The first implementation step is to create a clean development environment.

The agent should establish:

- repository structure,
- dependency management,
- configuration,
- environment variables,

- application entry point,
- test infrastructure,
- linting,
- formatting,
- basic CI if appropriate,
- README,
- development commands.

A typical AI application might look like:

```
app/
  api/
  agents/
  context/
  tools/
  models/
  retrieval/
  evaluation/
  storage/
  ui/

tests/
  unit/
  integration/
  evaluation/

scripts/
configs/
docs/
```

The exact structure is less important than establishing clear boundaries.

The architecture should be reflected in the repository.

5. Build the Vertical Slice First

A common mistake is asking the coding agent to implement the entire architecture before anything works.

Instead, build a **vertical slice**.

A vertical slice implements the smallest complete path through the system:

Input → Core Logic → Output

For an AI research assistant:

Question → Retrieval → LLM → Cited Answer

For an incident investigator:

Incident ID → Data Retrieval → Agent → Evidence-backed Report

The vertical slice proves that the architecture is viable.

Only after it works should additional capabilities be layered on.

6. Why Vertical Slices Matter

AI systems have many possible failure points:

UI → API → Orchestrator → Retrieval → Tool → LLM → Output

If all components are built simultaneously, debugging becomes difficult.

Suppose the final answer is wrong.

Where is the problem?

- UI?
- API?
- context construction?
- retrieval?
- tool?
- prompt?
- model?
- parser?
- verifier?

A vertical slice makes failures localizable.

The development strategy becomes:

Small Working System → Expand → Measure → Expand

rather than:

Build Everything → Discover It Doesn't Work

7. Agent Delegation

The coding agent should receive tasks at an appropriate level of abstraction.

Poor instruction:

“Fix the application.”

Better:

“Implement the incident retrieval service described in the specification. It should accept an incident ID, retrieve incident metadata, return a typed IncidentContext object, handle upstream failures explicitly, and include unit tests for success, missing incident, timeout, and malformed response cases.”

The second task contains:

- scope,
- interface,
- behavior,
- failure cases,
- tests.

This dramatically reduces ambiguity.

A useful task formulation is:

Task = Goal, Inputs, Outputs, Constraints, AcceptanceCriteria

8. Give the Agent Boundaries

A powerful coding agent can make large changes.

That makes boundaries important.

Explicitly define:

Files it may modify For example:

```
app/agents/  
app/tools/  
tests/
```

Files it should not modify For example:

```
infrastructure/  
production configuration/  
secrets/
```

Dependencies it may introduce Avoid uncontrolled dependency growth.

APIs it may call Restrict external access.

Commands it may execute Avoid dangerous or unnecessary operations.

The principle is:

$$\text{Agent Autonomy} \leq \text{Authorized Action Space}$$

This is the same principle used for production agents.

A coding agent is itself an agentic system.

9. Observe the Agent's Work

Do not treat the coding agent as a black box.

Observe:

- files changed,
- dependencies added,
- commands executed,
- tests created,
- tests passed,
- warnings,
- architectural deviations,
- assumptions made.

The agent's output is not merely code.

It is evidence about the agent's interpretation of the specification.

If the agent repeatedly misunderstands a requirement, the problem may not be the implementation.

The specification may be ambiguous.

This creates a useful feedback loop:

Agent Behavior $\rightarrow$ Specification Feedback

The coding process therefore becomes a test of the specification itself.

10. Test Continuously

Do not wait until the entire system is built before testing.

After each meaningful implementation step:

Implement $\rightarrow$ Test $\rightarrow$ Inspect

The testing hierarchy should include:

Unit tests Test isolated components.

$$f(x) \rightarrow y$$

Integration tests

Test component interactions.

$$A \rightarrow B \rightarrow C$$

End-to-end tests

Test complete user workflows.

$$\text{User} \rightarrow \text{System} \rightarrow \text{Outcome}$$

AI evaluations

Test probabilistic behavior against a golden dataset.

$$D \rightarrow \text{System}(D) \rightarrow \text{Metrics}$$

These layers answer different questions.

11. Deterministic Tests Are Not Enough

Traditional software testing assumes deterministic behavior.

AI systems violate that assumption.

A function might behave like:

$$f(x) = y$$

An LLM system behaves more like:

$$P(y|x)$$

Therefore, testing must evaluate distributions of outcomes and properties of outputs.

For example, instead of asserting:

“The answer must equal this exact string.”

test:

- Is the answer factually correct?
- Is it grounded in retrieved evidence?
- Does it contain required fields?
- Does it cite the correct sources?
- Does it express uncertainty appropriately?
- Did the system use the correct tools?

This is the difference between:

Output Equality

and:

Behavioral Correctness

12. Build the Evaluation Harness During the Build

The evaluator role should not begin after implementation.

Build evaluation infrastructure alongside the product.

A minimal harness might look like:

$$D_{\text{golden}} \rightarrow AI\ System \rightarrow \hat{Y} \rightarrow Evaluator \rightarrow Metrics$$

For each test case, record:

- input,
- expected properties,
- actual output,
- evidence retrieved,
- tool calls,
- latency,
- token usage,
- cost,
- evaluation score.

This creates an empirical development loop.

13. The Agent Should Test Its Own Work

An important pattern is:

$$\text{Generate} \rightarrow \text{Test} \rightarrow \text{Observe Failure} \rightarrow \text{Fix}$$

The coding agent can often perform this loop autonomously.

For example:

Implement the feature, run the relevant tests, inspect failures, fix implementation issues, and rerun the tests.

This allows the agent to perform multiple implementation iterations without requiring the human to micromanage each one.

However, there is an important constraint:

Passing tests does not prove that the product is correct.

The agent can optimize for the tests you gave it.

If the tests encode the wrong assumptions, the agent can produce a perfectly tested wrong system.

Therefore:

Automated Verification $\neq$ Product Judgment

14. Human Evaluation Remains Necessary

The product owner should periodically inspect real outputs.

Look for:

- incorrect assumptions,
- poor UX,
- unnecessary complexity,
- hallucinations,
- misleading confidence,
- workflow friction,
- missing information,
- inappropriate autonomy.

Ask:

“Would I actually use this?”

and:

“Does this solve the original problem?”

These questions cannot always be answered by unit tests.

The evaluation stack therefore becomes:

Automated Tests + AI Evals + Human Evaluation + User Outcome Metrics
--

15. Architecture Review

The architect role requires periodically stepping back from implementation details.

Ask:

Is the architecture still aligned with the specification?

Has the agent introduced unnecessary abstractions?

Are boundaries clear?

Is state managed correctly?

Are tools isolated?

Is context constructed deliberately?

Are model calls observable?

Are failures recoverable?

Is the system unnecessarily coupled to one provider?

Can individual components be evaluated independently? AI coding agents frequently over-engineer.

They may introduce:

- unnecessary frameworks,
- excessive abstraction layers,
- generic agent architectures,
- premature plugin systems,
- elaborate configuration systems,
- unnecessary dependencies.

The correct response is not to accept complexity merely because the agent produced it.

The architect must enforce:

$$\boxed{\text{Complexity} \leq \text{Problem Complexity}}$$

16. Avoid Premature Generalization

One of the most common agent-generated design problems is building for hypothetical future requirements.

For example:

“We may eventually support five model providers, twenty tools, multiple agents, and several data sources.”

The agent may respond by creating a generalized abstraction framework.

For an MVP, this is usually unnecessary.

Prefer:

Concrete Working System

over:

General Framework for Future Systems

Generalize when there is evidence that generalization is required.

The principle is:

Abstract from observed repetition, not imagined repetition.

17. Manage Context Deliberately

Coding agents have context limitations just like application agents.

A large repository can overwhelm the agent.

The agent may need:

- architecture documentation,
- repository maps,
- relevant source files,
- tests,
- interface definitions,
- recent changes.

A useful context hierarchy is:

$$C = C_{\text{architecture}} + C_{\text{task}} + C_{\text{relevant code}} + C_{\text{tests}} + C_{\text{constraints}}$$

Do not provide the entire repository indiscriminately.

The goal is:

Maximum Relevant Context

rather than:

Maximum Context

This is the same context-engineering principle applied to software development.

18. Git Is Part of the Control System

Version control becomes particularly important when agents can make large changes quickly.

Use small, meaningful commits.

A useful sequence is:

$$Commit_1 \rightarrow Commit_2 \rightarrow Commit_3$$

where each commit corresponds to a coherent change.

This creates:

- rollback points,
- auditability,
- easier debugging,
- comparison between implementations,
- safer experimentation.

The agent should not be allowed to turn the repository into an opaque sequence of thousands of modifications.

A useful principle is:

Agent Speed $\Rightarrow$ Greater Need for Reversibility

19. The Build Loop

The complete Chapter 25 loop can be represented as:

Specify $\rightarrow$ Delegate $\rightarrow$ Implement $\rightarrow$ Test $\rightarrow$ Evaluate $\rightarrow$ Inspect $\rightarrow$ Revise

Repeat until the MVP satisfies the acceptance criteria.

The human should spend most of their time on the high-leverage portions of this loop:

Direction Constraints Judgment Evaluation

rather than typing implementation details.

20. A Practical Build Sequence

A full build day can follow this sequence.

Phase 1 — Repository setup

Establish:

- repository,
- dependencies,
- configuration,
- test framework,
- application skeleton.

Phase 2 — Core data model

Implement:

- domain objects,
- schemas,
- persistence,
- interfaces.

Phase 3 — Vertical slice

Build:

Input → Core AI Workflow → Output

Phase 4 — Tool integration

Connect the required external systems.

Phase 5 — Context engineering

Implement:

- retrieval,
- context construction,
- state,
- prompt assembly.

Phase 6 — Agent behavior

Implement:

- planning,
- tool selection,
- execution,
- stopping conditions.

Phase 7 — Verification

Implement:

- output validation,
- evidence checking,
- error handling,
- confidence/uncertainty mechanisms.

Phase 8 — Evaluation

Run:

- unit tests,
- integration tests,

- golden datasets,
- AI evaluations.

Phase 9 — UX

Make the primary workflow usable by the target persona.

Phase 10 — Hardening

Address:

- security,
- logging,
- observability,
- latency,
- cost,
- failure recovery.

This sequence deliberately prioritizes a working path over architectural completeness.

21. Cost Is a First-Class Build Constraint

AI systems have variable inference costs.

A naive agent may perform:

$$N_{\text{calls}} \gg 1$$

LLM calls, retrieval operations, and tool calls can accumulate quickly.

A simple cost model is:

$$C_{\text{request}} = C_{\text{LLM}} + C_{\text{retrieval}} + C_{\text{tools}} + C_{\text{infrastructure}}$$

For an agent:

$$C_{\text{agent}} = \sum_{i=1}^N C_i$$

where N is the number of model/tool operations.

During the build, measure:

- tokens,
- model calls,
- latency,
- external API calls,
- storage,
- compute.

Do not optimize prematurely.

But do not leave cost completely unmeasured.

An MVP that works but costs \$50 per user action may not be a viable product.

22. Reliability Is a System Property

A model's benchmark score does not determine the reliability of the product.

Overall system reliability depends on multiple components:

$$R_{\text{system}} \approx R_{\text{retrieval}} \times R_{\text{tools}} \times R_{\text{model}} \times R_{\text{verification}} \times R_{\text{orchestration}}$$

This multiplicative intuition is useful.

If any critical component is unreliable, the overall system can become unreliable.

For example:

- excellent model,
- poor retrieval,

can still produce poor answers.

Or:

- excellent reasoning,
- unreliable tools,

can still produce a failed workflow.

The system must therefore be evaluated end-to-end.

23. Failure Handling

AI-native systems should assume that failure is normal.

Possible failures include:

- model timeout,
- malformed output,
- hallucination,
- unavailable tool,
- retrieval failure,
- conflicting evidence,
- context overflow,
- authentication failure,
- rate limiting,
- unexpected user input.

The architecture should define:

$$F_i \rightarrow \text{Detection} \rightarrow \text{Recovery} \rightarrow \text{Escalation}$$

For example:

$$\text{Tool Failure} \rightarrow \text{Retry} \rightarrow \text{Alternative Source} \rightarrow \text{Human Notification}$$

The system should not silently continue when a critical dependency fails.

24. Human Control

The product owner must define where autonomy ends.

A useful model is:

$$\text{AI Recommendation} \rightarrow \text{Human Approval} \rightarrow \text{Action}$$

For the MVP, keep the agent within a narrow permission boundary.

For example:

Allowed

- read logs,
- search tickets,
- inspect deployments,
- generate analysis.

Not allowed

- modify production systems,
- deploy code,
- restart services,
- delete data.

This establishes a safe progression:

$$\text{Read} \rightarrow \text{Recommend} \rightarrow \text{Approve} \rightarrow \text{Act}$$

Autonomy can increase later as evidence justifies it.

25. The Agent as a Junior Engineer

A useful mental model is to treat the coding agent as an extremely fast but imperfect junior engineer.

It can:

- implement quickly,
- follow explicit instructions,
- generate tests,
- search documentation,
- fix obvious errors.

But it may also:

- misunderstand requirements,
- make incorrect assumptions,
- over-engineer,
- miss edge cases,
- produce plausible but incorrect code,
- optimize for passing tests rather than satisfying the product.

Therefore:

$$\boxed{\text{Agent Output} = \text{Proposal}}$$

not:

$$\boxed{\text{Agent Output} = \text{Truth}}$$

The human architect must review consequential decisions.

26. The Agent as a Compiler for Intent

A more powerful mental model is to treat the coding agent as a compiler.

You provide:

Product Intent + Constraints + Architecture

and the agent produces:

Executable Software

Conceptually:

$$Compiler_{AI} : S \rightarrow I$$

where:

- S = high-level specification,
- I = implementation.

But unlike a traditional compiler, the transformation is probabilistic:

$$P(I|S)$$

This explains why evaluation and iterative refinement are necessary.

A traditional compiler either compiles correctly or fails.

An AI coding agent can produce a syntactically valid implementation that is semantically wrong.

Therefore:

$$\boxed{\text{AI Coding} = \text{Generation} + \text{Verification}}$$

27. Build Metrics

During Chapter 25, measure the development process itself.

Useful metrics include:

Implementation velocity How much functionality was implemented per unit time?

Agent success rate How often does the agent complete tasks without substantial human intervention?

Rework rate How much generated code must be rewritten?

$$R_{\text{rework}} = \frac{\text{Reworked Code}}{\text{Generated Code}}$$

Test effectiveness

How many meaningful defects are caught automatically?

Specification quality How often does the agent misunderstand requirements?

Human intervention How much engineering time is required to guide the agent?

These metrics help determine whether AI coding is actually improving engineering productivity.

28. The Real Objective of Chapter 25

The objective is not:

“Have an AI agent write as much code as possible.”

It is:

Learn how to build reliable software by delegating implementation to an AI system while retaining human control over product intent, architecture, and evaluation.

This distinction matters.

If the agent writes 10,000 lines of code but nobody understands whether the system solves the user's problem, the build failed.

If the agent writes 2,000 lines and produces a working, evaluated MVP that users genuinely value, the build succeeded.

Therefore:

$$\boxed{\text{Engineering Productivity} \neq \text{Lines of Code}}$$

A better measure is:

$$\text{Productive Engineering} = \frac{\text{Validated User Value}}{\text{Human Time}}$$

AI coding agents have the potential to dramatically increase this ratio.

29. Chapter 25 Deliverable

At the end of the day, you should have:

Working application A functioning MVP implementing the core workflow.

Source repository With:

- clean structure,
- configuration,
- tests,
- documentation.

Evaluation harness Including:

- golden dataset,
- automated metrics,
- representative test cases.

Architecture Documented and consistent with the implementation.

Metrics Including:

- latency,
- cost,
- AI quality,
- workflow success.

Known limitations A written list of:

- failures,
- missing functionality,
- unreliable behaviors,
- technical debt,
- product assumptions.

Demo A complete end-to-end demonstration:

User → Intent → AI System → Tools → Output → Evaluation

The system should work on at least one realistic scenario from beginning to end.

30. The Chapter 25 Build Checklist

Before declaring the MVP complete, verify:

- ☐ The primary user workflow works end-to-end.
 - ☐ The implementation matches the Chapter 24 specification.
 - ☐ The repository has a coherent architecture.
 - ☐ Core components have unit tests.
 - ☐ Critical integrations have integration tests.
 - ☐ The primary workflow has an end-to-end test.
 - ☐ The AI system has a golden evaluation set.
 - ☐ Outputs are evaluated for correctness and grounding.
 - ☐ Tool calls are observable.
 - ☐ Model calls are observable.
 - ☐ Errors are surfaced rather than silently ignored.
 - ☐ AI permissions are explicitly bounded.
 - ☐ Sensitive configuration is not embedded in source code.
 - ☐ Latency is measured.
 - ☐ AI/inference cost is measured.
 - ☐ The MVP scope has not expanded uncontrollably.
 - ☐ Known limitations are documented.
 - ☐ A real user can complete the core task.
 - ☐ The product hypothesis can now be tested with evidence.
-

31. Key Takeaways

1. **Chapter 25 is the transition from specification to execution.** The product has been defined; now the system must be built.

2. **The coding agent should perform most implementation work.**
The human’s leverage comes from directing, constraining, inspecting, and evaluating the agent.
3. **Your role becomes Product Owner + Architect + Evaluator.**
You own the “what,” “why,” constraints, architecture, and definition of correctness.
4. **Start with a vertical slice.** Build one complete path through the system before expanding functionality.
5. **Treat the specification as the source of truth.** If the agent is repeatedly making product decisions, either the specification is incomplete or the agent is exceeding its role.
6. **Agent autonomy requires explicit boundaries.** Define what the coding agent may modify, execute, install, and access.
7. **Test continuously.** The development loop should be:

Implement → Test → Inspect → Revise

8. **AI systems require more than conventional tests.** Combine deterministic tests, AI evaluations, human evaluation, and user outcome metrics.
9. **Passing tests does not prove product correctness.** The coding agent can optimize for the tests it receives. The product owner must evaluate whether the system actually solves the intended problem.
10. **Keep the architecture proportional to the MVP.** Avoid premature abstraction and speculative generalization.
11. **Measure cost, latency, reliability, and intervention.** A technically functional AI system may still be economically or operationally unusable.
12. **Use version control as a control mechanism.** Faster AI-generated changes increase the value of small commits, auditability, and rollback.
13. **Treat agent output as a proposal, not truth.** AI coding agents are powerful implementation systems but remain probabilistic.
14. **The deepest shift is in the engineering abstraction level.**

Traditional development:

Human → Code → Software

AI-native development:

Human Intent → Specification → Coding Agent → Implementation → Evaluation

15. **The objective is not to maximize generated code.** It is to maximize validated user value per unit of human engineering effort.

$$\text{Productive Engineering} = \frac{\text{Validated User Value}}{\text{Human Effort}}$$

Chapter 25 therefore represents an important transition in the AI engineering discipline: **the engineer increasingly becomes the designer and governor of intelligent software-production systems, rather than the person who manually produces every implementation detail.**